

네트워크 게임프로그래밍 텀프로젝트

8팀 추진계획서

2020180029	이준호
2020112036	정현도
2020182045	황수지

목차

01. 기획	3
1) 개요	3
2) 게임 요소 및 시스템	3
3) 개발 환경	6
01. High-level 디자인	7
1) 게임 흐름도	7
2) 서버 - 클라이언트 구조	7
3) 클라이언트 구조	8
4) 서버 구조	9
02. Low-level 디자인	11
1) 서버 - 클라이언트	11
2) 서버	13
3) 클라이언트	14
03. 역할 분담	15
04. 개발 일정	16

01. 기획

1) 개요

이름	Dark Knight
장르	플랫폼 게임 (횡스크롤 액션 게임)
내용	플레이어는 다양한 무기를 사용해 몬스터를 처치하고, 장애물을 파괴하며 점수를 획득해 스테이지를 클리어하는 것이 목표.
제작 교과목	2023 윈도우 프로그래밍 (정현도, 이준호 제작)

2) 게임 요소 및 시스템

- 플레이어

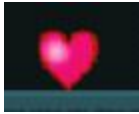
① 조작 방법

좌우 이동	방향키 ←, →
포탈 출입	방향키 ↑
점프	SPACE
무기 변경	TAP
공격	A

② 역할 선택 (검사/궁수)

검사		근거리 공격 담당 + 트랩 제거 기능
궁수		원거리 공격 담당

- 아이템

포션		HP 회복
스타		트랩 제거 시 생성 공격력 향상

- 장애물

트랩		닿으면 데미지 입는다 근거리 공격 시 제거됨
----	---	-----------------------------

- 몬스터 종류

근거리 몬스터		좌우로 움직이고, 닿으면 데미지를 입음
원거리 몬스터		가만히 서 있는 상태에서 원거리 공격을 한다. 공격에 맞으면 데미지 입음. 공격을 플레이어의 공격으로 상쇄 시킬 수 있음.
보스 몬스터	- 플레이어가 일정범위 안에 다가오면 자동적으로 공격함. - 일정 시간마다 3가지의 공격 패턴 중 랜덤하게 공격함. - 보스가 울음소리를 낼 때는 공격이 통하지 않음. <공격 패턴>   	① 플레이어가 일정범위 안에 다가오면 자동적으로 공격함. ② 플레이어의 방향으로 3개의 공격을 날린다. 플레이어는 이 공격을 피하거나 검으로 베어 회피할 수 있다. ③ 보스가 순간적으로 사라지며 일정시간 이후 플레이어 주변에 나타나 기본공격을 가한다.

- 스테이지 구성

스테이지1	 간단한 지형으로 장애물을 부숴가며 몬스터들을 처치해야함.
스테이지2	 맵은 기본적으로 간단하며 보스와 싸우는 장소에 공격을 피할 수 있도록 순간이동하는 포탈이 있는 발판이 있음. 보스를 물리치면 게임 클리어.

+) 추가 요소

역할 선택 및 게임 준비 기능	Player1  준비중 Player2  준비완료 게임 시작
------------------	--

3) 개발 환경

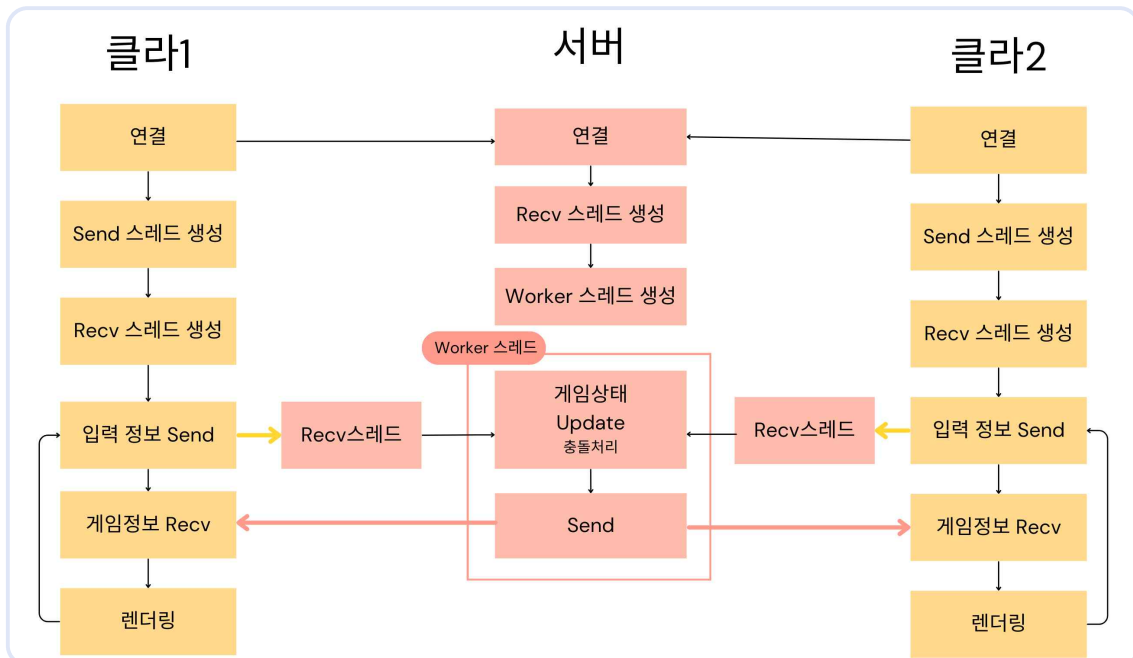
- 개발 도구: Visual Studio 2022
- 운영 체제: Windows 11
- API: Windows API (WinAPI)
- 네트워크 프로토콜: TCP/IP
- 네트워크 라이브러리: WinSock
- 개발 언어: C/C++
- 협업 도구: GitHub

02. High-level 디자인

1) 게임 흐름도

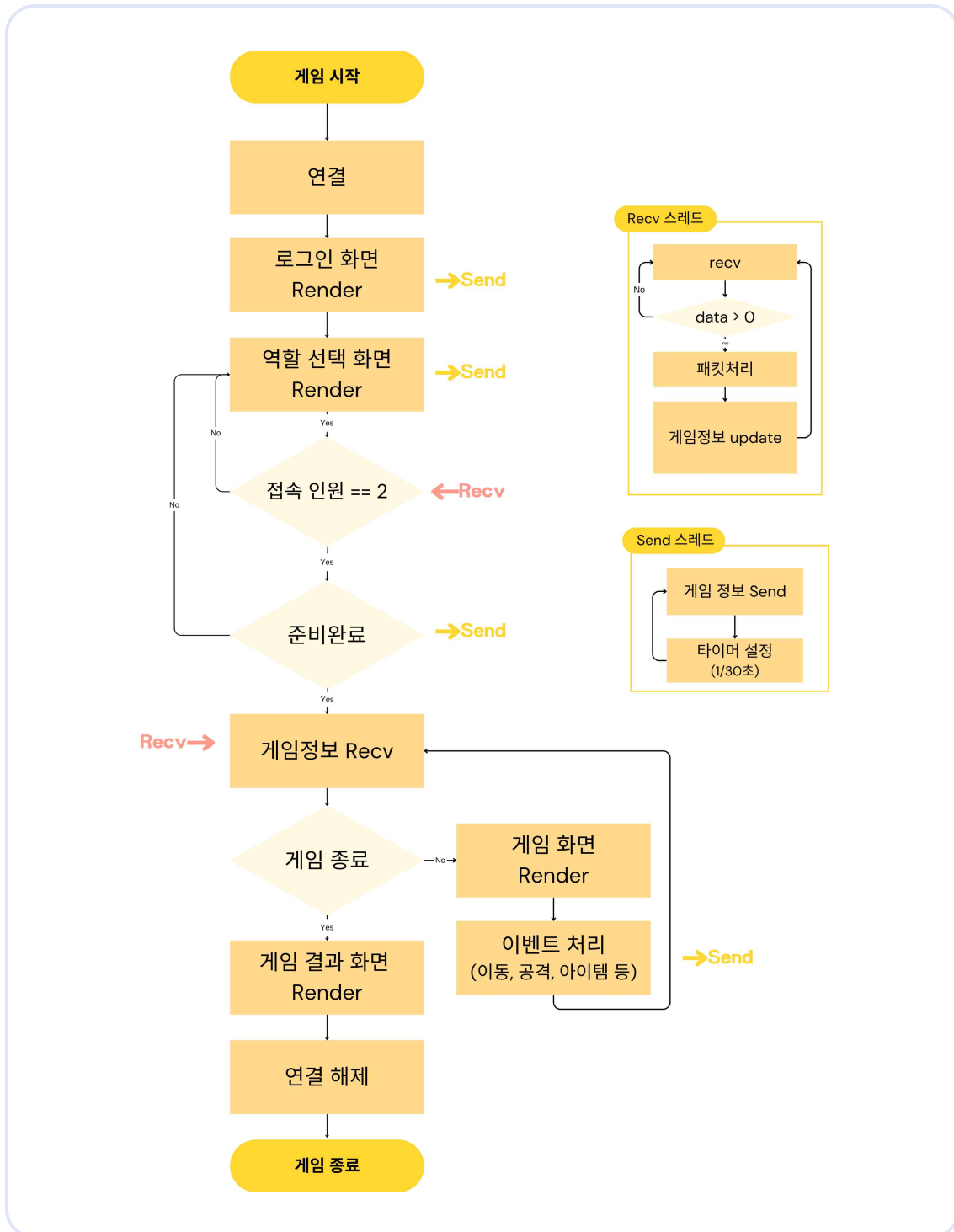


2) 서버 - 클라이언트 구조

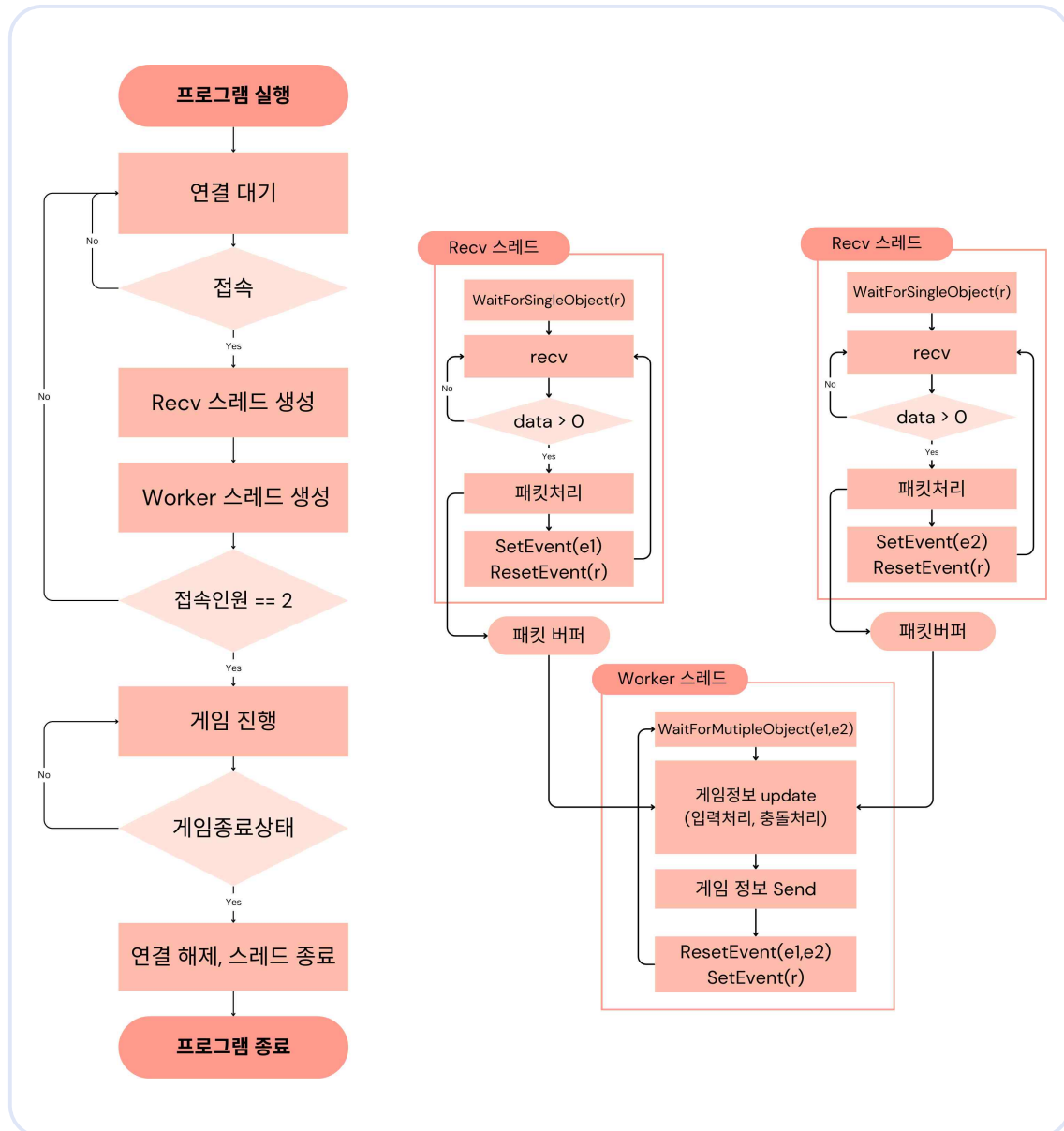


- 클라이언트는 서버가 가진 동일한 게임정보를 받아 업데이트하고 렌더링해서 동기화한다.
- 서버는 클라이언트의 입력정보에 따라 이동 및 충돌처리 작업을 하고 게임정보를 업데이트한다.
- 서버는 업데이트 된 게임정보를 클라이언트 모두에게 전송한다.

3) 클라이언트 구조

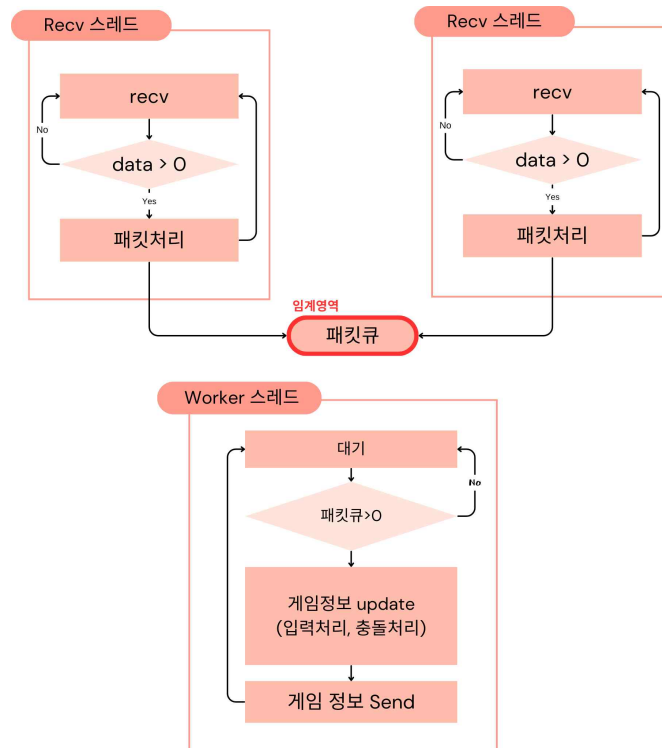


4) 서버 구조



- 접속되면 Recv, Send 스레드를 통해 게임 진행 전 로그인과 게임 준비 상태를 주고받는다.
- 두 클라이언트가 접속이 완료되면 게임 진행에 필요한 정보한 정보를 주고받는다.
- 게임 상태가 종료상태가 된다면 스레드를 종료하고 연결을 해제한다.

+) 서버 스레드 구조 다른 방법



03. Low-level 디자인

1) 서버 - 클라이언트

① 구조체 타입

PlayerInfo		
타입	멤버변수명	역할
int	id	식별 번호
int	x, y	위치
short	hp	체력
short	damage	공격력
char	type	검사/궁수 역할
char	state	Idle/점프/공격

MonsterInfo		
타입	멤버변수명	역할
int	id	식별 번호
char	type	몬스터 종류
int	x, y	위치
short	hp	체력
short	damage	공격력
char	direct	공격/이동 방향
char	attack_type	공격 패턴 (보스의 경우)

TileInfo		
타입	멤버변수명	역할
short	type	맵 이미지 종류
int	x, y	위치

ItemInfo		
타입	멤버변수명	역할
char	type	종류 (포션/스타/장애물)
int	x, y	위치
short	amount	아이템의 능력치

② 패킷 타입

Packet		
타입	멤버변수명	역할
char	type	패킷 종류
int	length	패킷 사이즈
vector<char>	data	패킷에 전송할 정보들

PacketType		
열거자	설명	전송자
InputKey	Input정보 전송 패킷	Client
LoginRequest	로그인 요청	Client
PlayerChoice	역할 선택 정보 패킷	Client
GameStartRequest	게임 준비 완료 시 게임 시작 요청	Client
PlayerMove	플레이어 이동 요청	Client
PlayerAttack	플레이어 공격 요청	Client
GameStartResponse	방 정보 상태 업데이트 및 맵 정보 제공	Server
GameState	게임 상태 패킷	Server
GameEndNotification	게임 종료 정보 처리 및 제공	Server
PlayerInfoUpdate	플레이어 이동 및 상태 업데이트	Server
MonsterInfoUpdate	몬스터 이동 및 상태 업데이트	Server

2) 서버

① 구조체

Client		
타입	멤버변수명	역할
PlayerInfo	player	플레이어 정보저장
int	gamestate	클라이언트가 속한 방 정보
int	id	식별 넘버
char	stage	진행 중 스테이지 단계
int	total_time	클리어 시간
array<Object, 4>	objects	현재 맵의 오브젝트

② 함수

타입	함수명	인자	기능
DWORD	WorkerThread	LPVOID arg	게임 정보 업데이트와 send 작업을 하는 스레드
DWORD	RecvThread	LPVOID arg	recv 작업을 하는 스레드
void	SendPlayerInfo	int PlayerID	플레이어 정보를 전송하는 함수
void	SendGameInfo	int PlayerID	게임 정보를 전송하는 함수
void	SendGameResult	int PlayerID	게임 종료 요청에 결과 보내는 함수
void	SendMonsterInfo	int PlayerID	몬스터 정보를 전송하는 함수
void	ProcessCollsion	int PlayerID	충돌 처리하고 충돌 결과 리턴
bool	IsCollisionMonster	int PlayerID	몬스터와 충돌 검사
bool	IsCollisionTile	int PlayerID	타일과 충돌 검사
bool	IsCollisionItem	int PlayerID	아이템과 충돌 검사
bool	IsCollisionTrap	int PlayerID	장애물과 충돌 검사

3) 클라이언트

Scene		
타입	멤버변수명	역할
vector<RECT>	button	버튼
int	num	식별번호

LoginScene : public Scene		
타입	멤버변수명	역할
char*	id	서버로 보낼 id

RoomScene : public Scene		
타입	멤버변수명	역할
PlayerInfo	player[2]	플레이어 저장

GameScene : public Scene		
타입	멤버변수명	역할
array<vector<Object>, 4>	objects	타일 맵, 적 객체, 아이템, 트랩
PlayerInfo	player[2]	플레이어 저장

ClientNetwork			
타입	멤버변수명	역할	
SOCKET	socket	통신을 위한 클라이언트의 소켓	
타입	함수명	인자	기능
DWORD	SendThread	LPVOID arg	send 작업을 하는 스레드
DWORD	RecvThread	LPVOID arg	recv 작업을 하는 스레드
void	SendLoginRequest	char* id	로그인 요청
void	SetUserData		유저데이터를 받음
void	ChangeScene		씬 변경
void	SendChangeCharacter		캐릭터 정보 변경을 요청
void	SendReadyState	bool isReady	플레이어 준비 정보를 보냄
void	RecvRoomData		방 정보, 플레이어 정보를 받고 업데이트함
void	RecvGameSceneData		게임씬 정보를 받음
void	SendPlayerData		플레이어 정보, 키 입력을 보냄
void	RecvPlayerData		플레이어 정보를 받음
void	SetPlayerData		받은 플레이어 정보를 적용함
void	SetGameSceneData		받은 씬 정보를 적용시킴
void	SendQuitScene		퇴장 정보를 보냄
void	SendInputKey	UINT message, WPARAM wParam, LPARAM lParam	입력 정보를 보냄

04. 역할 분담

역할	할 일	정현도	이준호	황수지
서버	클라이언트와 연결 및 Recv, Send 스레드 생성			
	로그인 정보 패킷 처리 및 클라이언트 동기화			
	역할 선택 정보 패킷 처리 및 클라이언트 동기화			
	게임 상태 판단 및 패킷 클라이언트로 전송			
	플레이어의 이동 처리 및 클라이언트 동기화			
	몬스터 위치와 생존 유무 패킷 전송			
	충돌처리를 위한 클래스를 구현			
	오브젝트와의 충돌 처리 구현 및 패킷 전송			
	스테이지 클리어 처리 구현 및 스테이지 맵정보 패킷 전송			
	플레이어 공격에 따른 몬스터 상태 업데이트 구현 및 전송			
	플레이 시간 정보 구현 및 패킷 전송			
	게임 종료 여부 판단 함수 구현 (플레이어 상태, 몬스터 처리)			
	게임 결과 패킷 전송			
클라이언트	오브젝트 클래스 구현			
	서버와 연결			
	플레이어 입력 패킷 전송			
	플레이어 위치 정보 패킷 처리 및 렌더링			
	플레이어 위치에 따라 카메라 스크롤링 구현			
	로그인 화면을 구현 및 로그인 패킷 전송			
	역할 선택 화면 구현 및 역할 선택 정보 패킷 전송			
	게임 준비 상태 구현 및 패킷 전송			
	서버로부터 맵 정보 패킷 처리 및 렌더링			
	플레이어 상태 정보 패킷 처리 및 렌더링			
	몬스터 상태 정보 패킷 처리 및 렌더링			
	플레이어 사망시 생존한 플레이어 시점으로 카메라 전환 기능 구현			
	플레이 시간 정보 패킷 처리 및 렌더링			
	게임 결과 패킷 처리 및 렌더링			
기타	추가 리소스 제작 (캐릭터 선택창, ID 입력 화면)			
	패킷 프로토콜 설계 및 제작			
	문서 정리			

05. 개발 일정 (11/3 ~ 12/5)

주	일 3	월 4	화 5	수 6	목 7	금 8	토 9
1						패킷 프로토콜 설계 및 제작	오브젝트 클래스 구현 (플레이어)
						Protocol.h Define.h	CPlayer CArcher
						패킷 프로토콜 설계 및 제작	서버와 연결
						Protocol.h Define.h	CNetworkMa nager Init() Connect() CleanUp()
						패킷 프로토콜 설계 및 제작	클라이언트와 연결 Recv, Worker스레드 생성
						Protocol.h Define.h	RecvTread() WorkerTread ()
2	10	11	12	13	14	15	16
	오브젝트 클래스 구현 (아이템, 장애물)	플레이어 정보 전송			로그인 화면을 구현 및 로그인 패킷 전송	역할 선택 화면 구현 및 역할 선택 정보 패킷 전송	접속 인원수와 준비 상태를 판단해 게임 상태를 진행 상태로 변경 및 게임 상태 패킷 클라이언트로 전송
	CItem CTrap	SendPlayerData()			SendLoginR equest()	SendChange Character()	IsAllready(), SendRoomIn fo()
	오브젝트 클래스 구현 (몬스터)	오브젝트 클래스 구현 (타일)	플레이어 위치에 따라 카메라 스크롤링 구현			역할 선택 정보 패킷 처리 및 클라이언트 동기화	충돌처리 작업을 위한 클래스를 구현
	CGhost CRiche	CTile	CCamera			SendPlayerIn fo()	CCollision
		플레이어의 이동 패킷 처리 및 클라이언트 동기화		추가 리소스 제작 (캐릭터 선택창, ID 입력 화면)		로그인 정보 패킷 처리 및 클라이언트 동기화	게임 준비 상태 체크
		UpdatePlayer()				UpdateLogin Data()	UpdateRead y()

	17	18	19	20	21	22	23
3	오브젝트(플레이어-타일)와의 충돌 처리 구현	오브젝트(플레이어-장애물)와의 충돌 처리 구현			서버로부터 맵 정보 패킷 처리		플레이어 공격에 따른 몬스터 상태 업데이트 구현 및 전송
	IsCollisionTile()	IsCollisionTrap()			UpdateMapData()		IsAttackCollision()
		오브젝트(플레이어-몬스터)와의 충돌 처리 구현				서버로부터 맵 정보 렌더링	몬스터 상태 정보 패킷 처리 및 렌더링
		IsCollisionMonster()				RenderMap()	UpdateMonster()
		오브젝트(플레이어-아이템)와의 충돌 처리 구현		플레이어 상태 정보 패킷 처리		플레이어 렌더링	몬스터 렌더링
		IsCollisionItem()		UpdatePlayer()		RenderPlayer()	RenderMonster()
	24	25	26	27	28	29	30
4		플레이 시간 정보 구현 및 패킷 전송		플레이어 사망시 생존한 플레이어 시점으로 카메라 전환 기능 구현		게임 결과 패킷 처리	
		SendPlayTime()		ChangeCameraTarget()		UpdateGameResult()	
		스테이지 클리어 처리 구현		스테이지 맵정보 전송			게임 결과 패킷 전송
		IsStageClear()		SendStageData()			SendGameResult()
	아이템 획득에 따른 플레이어의 상태 패킷 전송		플레이 시간 정보 패킷 처리	플레이 시간 렌더링	게임 종료 여부 판단 함수 구현 (플레이어 상태, 몬스터 처치)		게임 결과 렌더링
	UpdatePlayerState()		UpdatePlayTime()	RenderPlayTime()	IsGameOver()		RenderGameResult()
5	1	2	3	4	최종발표		